

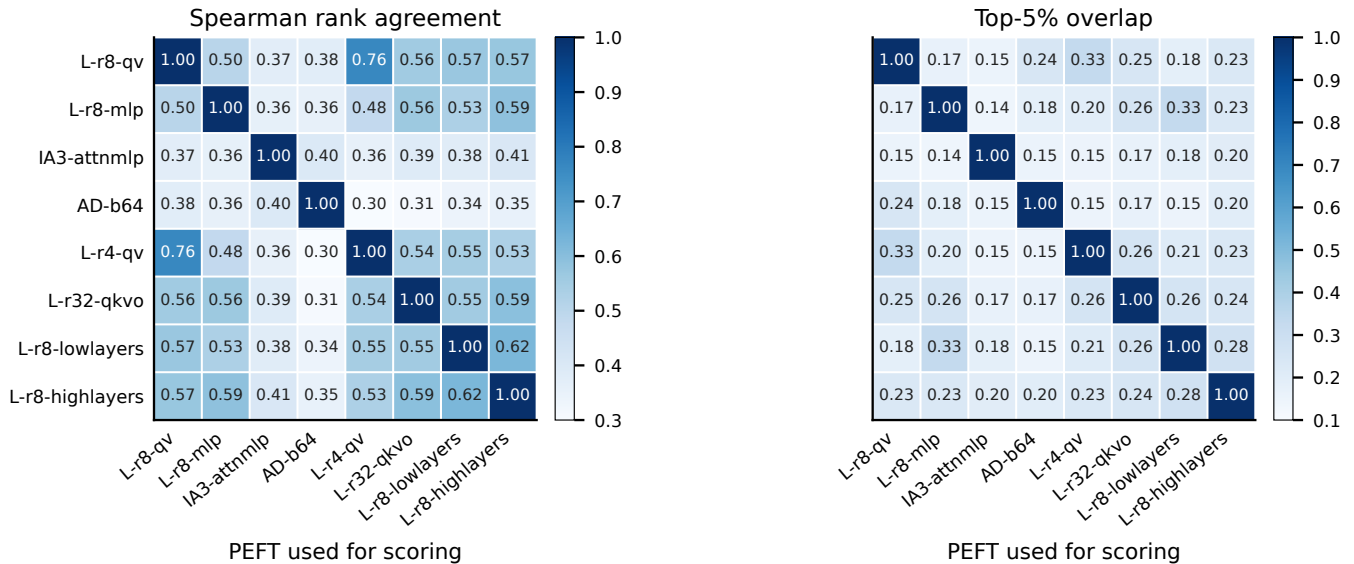


Figure 1: Short-update estimates—task-sketch loss reductions after a few PEFT updates—yield different data rankings across configurations. We score a common pool of 20K candidate examples using two task sketches and independent checkpoint/seed replicates. Each cell reports the Spearman rank correlation between its row and column runs; the diagonal compares each run with itself. Independent runs for the same PEFT reach $\rho = 0.79$ and Top-5% overlap = 0.32, providing a reference for variation in these labels. Cross-family pairs fall to $\rho = 0.36$ and Top-5% overlap = 0.17 on average.

cross-family mismatched selection by 2.0 points on average (Appendix Table 12). The same pattern appears within LoRA configurations, where matched selection improves performance by 1.45 points (§6).

These observations motivate *PEFT-conditioned utility*: the value of training on example x for task t under PEFT configuration p . We denote this value by $u(x, p, t)$. A task-only selector omits p and must assign the same ranking to every configuration. Per-PEFT influence selection includes p but recomputes gradients for each target. Our goal is to predict $u(x, p, t)$ while sharing the expensive computations used to learn these scores.

PCU-Select (PEFT-Conditional Utility) enables this sharing through a common representation. It maps every supported configuration to 24 *intervention sites*: selected Transformer module outputs through which PEFT updates affect the backbone. These outputs remain comparable even when PEFT methods train parameter tensors with different shapes. PCU-Select caches sample and task signals at the sites. A structured PEFT code records which sites a configuration affects, its update type, its capacity, and its training recipe. Supplying this code at scoring time preserves configuration information while keeping the cached signals reusable.

A conditional scorer turns the shared signals into a target-specific ranking. Its prediction depends on the candidate example, task sketch, and PEFT code. It learns from two complementary signals: inexpensive labels based on gradient alignment cover the full candidate pool, while short-update labels measure task-sketch loss reductions for a smaller sample. At deployment, the scorer ranks the cached examples for a target configuration and task. PCU-Select then spreads the budget across groups of semantically similar examples

to limit near-duplicates. This pipeline moves gradient extraction and label construction offline while retaining a distinct subset for each target.

These design choices amortize target-specific selection across a PEFT registry. We make three contributions:

- **PEFT-conditioned utility.** We formulate data value as $u(x, p, t)$ and show that both data-value rankings and downstream subset quality depend on the PEFT configuration (Figure 1; Appendix Table 12).
- **Intervention-site amortization.** PCU-Select represents different PEFT parameterizations through the same 24 module-output sites. A structured code preserves each target’s update locations, type, capacity, and recipe without rebuilding a gradient datastore. Removing the PEFT code or task sketch reduces downstream performance by 1.37 or 0.82 points, respectively (Appendix Table 11).
- **Registry-scale quality and cost.** Across four tasks and five seen PEFTs on Llama-2-7B (Touvron et al. 2023), PCU-Select gains 0.74 points over the PEFT-agnostic representation baseline RDS+ and differs from per-PEFT LESS by +0.04. It reduces the selection cost of adding one four-task PEFT by $44 \times$ (1.44 versus 63.2 GPU-hours; Tables 1 and 2).

2 Related Work

Instruction-data selection. Instruction-data selectors usually estimate quality or diversity independently of the target PEFT configuration. LIMA supports instruction tuning with 1,000 curated examples (Zhou et al. 2023). Model-based filters instead use judgments or instruction properties: Alpaca filters Alpaca examples with LLM judgments (Chen